

# Введение в Prisma ORM

Переходим от ручных SQL-запросов к работе с PostgreSQL через модели и Prisma Client

# Результаты занятия

---

- ✓ Объяснять назначение ORM
- ✓ Понимать место Prisma между Node.js и PostgreSQL
- ✓ Различать Prisma Schema, Prisma Client, Prisma Migrate и Prisma Studio
- ✓ Подключать Prisma к существующей базе shop\_db
- ✓ Получать модель Prisma из существующих таблиц через db pull
- ✓ Генерировать Prisma Client
- ✓ Выполнять findMany() и findUnique()
- ✓ Использовать Prisma внутри слоя Service

# Учебные вопросы

---

- 1 ORM и Prisma** Зачем нужен ORM и чем работа через Prisma отличается от ручного SQL
- 2 Подключение Prisma к проекту** Установка, DATABASE\_URL, конфигурация и анализ существующей базы
- 3 Prisma Schema и модели** Как таблицы, столбцы и связи представляются в Prisma
- 4 Prisma Client** Генерация клиента и первые запросы findMany() и findUnique()
- 5 Prisma в Express-приложении** Использование ORM в Product Service без изменения REST API



# Учебный вопрос 1. ORM и Prisma

Между JavaScript-кодом и реляционной базой появляется дополнительный программный слой

# Что такое ORM

---

- ORM — подход и набор инструментов, позволяющих работать с данными реляционной базы через объекты, модели и методы языка программирования.
- ORM не отменяет реляционную базу данных.
- Данные продолжают храниться в PostgreSQL.
- SQL-запросы всё равно выполняются на стороне СУБД.
- Разработчик для типовых операций использует API ORM.
- ORM преобразует операции приложения в запросы к базе данных.

# Ручной SQL и ORM

## node-postgres + SQL

- SQL пишется вручную
- Используется `pool.query()`
- Разработчик управляет параметрами `$1`, `$2`
- Результат находится в `result.rows`
- Полный контроль над SQL

## Prisma ORM

- Используются методы Prisma Client
- Модели описывают структуру данных
- Фильтры задаются JavaScript-объектами
- Результат сразу является JavaScript-объектом
- Типовые запросы требуют меньше SQL-кода

# Одна операция двумя способами

Запрос: Получить товар с id = 15

## Код • JAVASCRIPT

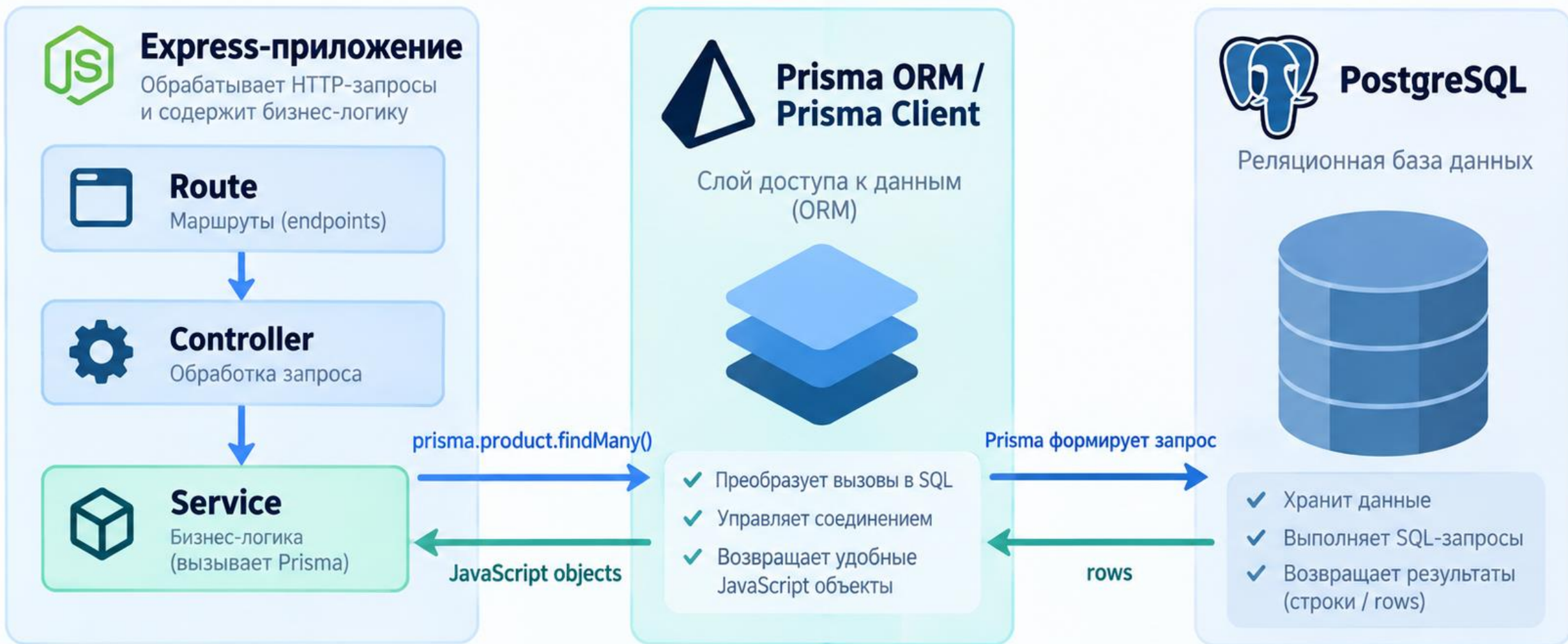
```
// node-postgres
const result = await pool.query(
  'SELECT * FROM products WHERE id = $1',
  [15]
);

// Prisma
const product = await
prisma.product.findUnique({
  where: { id: 15 }
});
```

## Пояснение

- Обе операции обращаются к PostgreSQL. Отличается интерфейс, который использует разработчик.

# Место Prisma ORM между приложением и PostgreSQL



**Prisma не заменяет PostgreSQL, а является **слоем доступа к данным**.**

# Основные части Prisma ORM

## ● Prisma Schema

Описание моделей, полей, ключей, связей и отображения на таблицы базы.

## ● Prisma Client

Сгенерированный API для выполнения запросов из Node.js.

## ● Prisma Migrate

Инструмент изменения структуры базы через миграции.

## ● Prisma Studio

Графический интерфейс для просмотра и редактирования данных.

# Какую версию используем в курсе

- В учебном проекте используем Prisma ORM 7.
- Prisma ORM 7 является стабильной и поддерживаемой веткой.
- Она подходит для Node.js 22.x и ESM.
- Prisma ORM 8 на момент подготовки материала находится на этапе Release Candidate.
- Использование стабильной версии уменьшает риск изменений CLI и API во время учебного курса.

## Главная идея

**Основные понятия ORM, модели и методы Prisma Client сохраняются независимо от последующего обновления версии.**

# Что ORM не делает автоматически

- Не заменяет понимание SQL.
- Не отменяет проектирование таблиц и связей.
- Не исправляет плохую структуру базы данных.
- Не гарантирует, что любой сформированный запрос будет эффективным.
- Не отменяет индексы, ограничения, транзакции и анализ производительности.
- Для сложных задач иногда по-прежнему нужен SQL.

## Главная идея

**ORM удобнее использовать после того, как разработчик понимает реляционную модель и основные SQL-команды.**



# Учебный вопрос 2. Подключение Prisma к проекту

Подключаем ORM к уже существующей PostgreSQL-базе shop\_db

# Наша исходная ситуация

---

- Express-приложение уже существует.
- PostgreSQL Server уже настроен.
- База shop\_db содержит реальные таблицы.
- Таблицы связаны через PRIMARY KEY и FOREIGN KEY.
- Product Service сейчас использует пакет pg и pool.query().
- Поэтому Prisma подключаем к существующей базе, а не создаём проект с нуля.

## Устанавливаем Prisma ORM 7

### BASH

```
npm install -D prisma@7 typescript tsx @types/node @types/pg  
npm install @prisma/client@7 @prisma/adapter-pg pg dotenv
```

# Назначение пакетов

| Пакет              | Назначение                                                  |
|--------------------|-------------------------------------------------------------|
| prisma             | CLI: init, db pull, generate, migrate, studio               |
| @prisma/client     | Prisma Client для запросов к базе                           |
| @prisma/adapter-pg | Адаптер Prisma для драйвера PostgreSQL                      |
| pg                 | Node.js-драйвер PostgreSQL                                  |
| dotenv             | Загрузка DATABASE_URL из .env                               |
| tsx                | Запуск проекта с ESM и генерируемым TypeScript-кодом Prisma |

# Инициализируем Prisma

**BASH**

```
npx prisma@7 init --datasource-provider postgresql --output ../generated/prisma
```

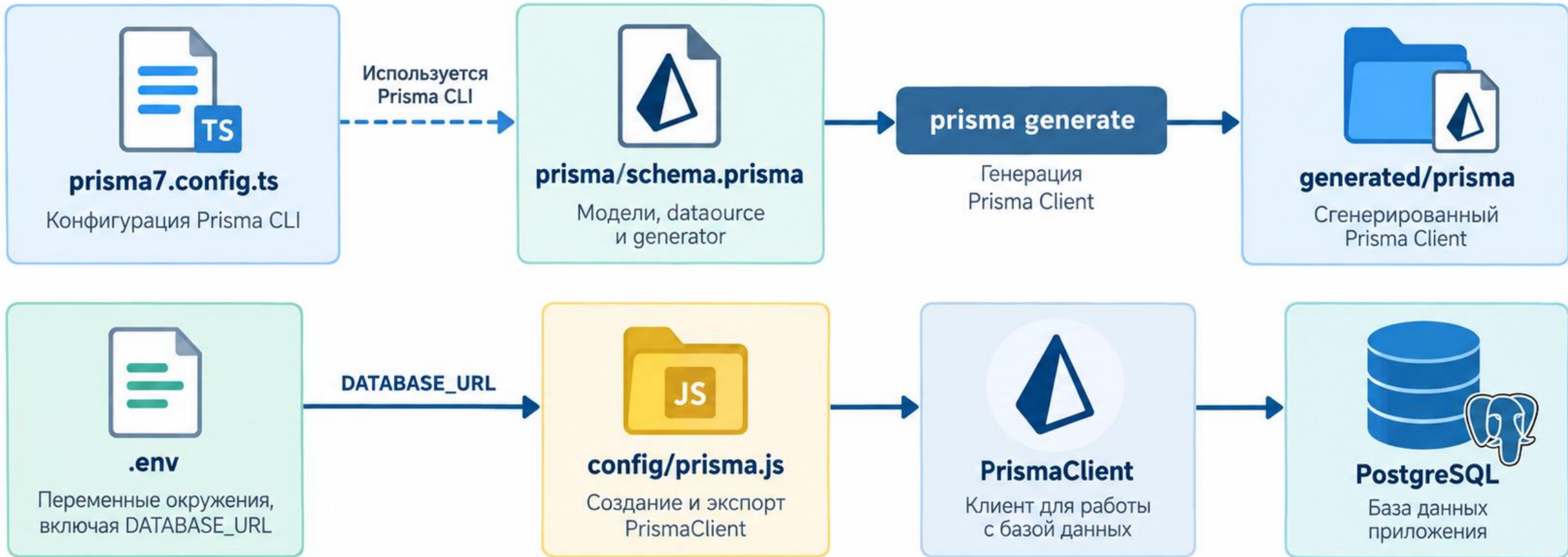
## Что появляется в проекте

### TEXT

```
express-shop/  
├── prisma/  
│   └── schema.prisma  
├── generated/  
│   └── prisma/ ← появится после generate  
├── prisma7.config.ts  
├── .env  
├── controllers/  
├── routes/  
├── services/  
├── config/  
└── app.js
```

# Файлы Prisma и их назначение

Как файлы связаны между собой и как Prisma работает в проекте



## Ключевые идеи:

- 1 **.env** — хранит переменные окружения (например, `DATABASE_URL`).
- 2 **prisma7.config.ts** — используется Prisma CLI / конфигурацией схемы.
- 3 **schema.prisma** — содержит модели, `datasource` и `generator`, используется для генерации клиента.
- 4 **prisma generate** — создает Prisma Client в папке `generated/prisma`.
- 5 **config/prisma.js** — создает и экспортирует PrismaClient, который подключается к PostgreSQL.

# Строка подключения DATABASE\_URL

## TEXT

```
DATABASE_URL="postgresql://shop_user:college123@localhost:5432/shop_db?schema=public"
```

## Что важно

- postgresql определяет тип СУБД.
- shop\_user — пользователь PostgreSQL.
- college123 — учебный пароль.
- localhost:5432 — адрес и порт сервера.
- shop\_db — существующая база интернет-магазина.
- schema=public указывает схему PostgreSQL.

# Конфигурация Prisma CLI

## TYPESCRIPT

```
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations"
  },
  datasource: {
    url: process.env.DATABASE_URL
  }
});
```

## Что важно

- CLI должен знать расположение Prisma Schema.
- DATABASE\_URL загружается из .env.
- Конфигурация используется командами Prisma.
- Подключение приложения во время выполнения настраивается отдельно через Prisma Client.

# Подключаем Prisma к существующей базе

- 1 Установить пакеты** Добавляем Prisma CLI, Prisma Client, PostgreSQL adapter и необходимые зависимости.
- 2 Выполнить prisma init** Создаём Prisma Schema и конфигурацию проекта.
- 3 Настроить DATABASE\_URL** Указываем существующую базу shop\_db.
- 4 Выполнить prisma db pull** Prisma анализирует существующие таблицы, поля, ключи и связи.
- 5 Выполнить prisma generate** На основе моделей генерируется Prisma Client.

# Анализируем существующую базу

## BASH

```
npx prisma@7 db pull
```

## Что важно

- Команда подключается к базе из DATABASE\_URL.
- Prisma читает структуру существующих таблиц.
- Анализируются столбцы, типы, ключи и связи.
- Результат записывается в schema.prisma.
- Существующие данные при db pull не удаляются.



# Учебный вопрос 3. Prisma Schema и модели

Реляционная схема PostgreSQL получает представление в виде моделей Prisma

# Что такое Prisma Schema

---

- Prisma Schema — декларативное описание моделей данных, их полей, ограничений, связей и способа генерации Prisma Client.
- Файл обычно называется `schema.prisma`.
- `model` описывает сущность.
- Поля модели соответствуют столбцам базы.
- Атрибуты начинаются с символа `@`.
- Связи также описываются через поля моделей.

# Простая модель Product

## PRISMA

```
model Product {  
  id          Int      @id @default(autoincrement())  
  name        String   @db.VarChar(150)  
  price       Int  
  categoryId  Int      @map("category_id")  
  inStock     Boolean  @default(true) @map("in_stock")  
  
  @@map("products")  
}
```

## Что важно

- Product — имя модели в приложении.
- id имеет тип Int и является первичным ключом.
- @default(autoincrement()) соответствует автоматическому id.
- @map связывает удобное имя JavaScript с именем столбца PostgreSQL.
- @@map связывает модель Product с таблицей products.

# Модель Category

## PRISMA

```
model Category {  
  id Int @id @default(autoincrement())  
  name String @unique @db.VarChar(100)  
  
  products Product[]  
  
  @@map("categories")  
}
```

## Что важно

- Category соответствует таблице categories.
- @unique соответствует ограничению UNIQUE.
- products содержит связанные объекты Product.
- Массив Product[] показывает сторону многих в связи 1:N.

## Связь Product → Category

### PRISMA

```
model Product {
  id          Int          @id @default(autoincrement())
  name        String       @db.VarChar(150)
  price       Int
  categoryId  Int          @map("category_id")
  inStock     Boolean      @default(true) @map("in_stock")

  category Category @relation(
    fields: [categoryId],
    references: [id]
  )

  @@map("products")
}
```

### Что важно

- categoryId представляет внешний ключ.
- category представляет связанную модель.
- fields указывает поле внешнего ключа.
- references указывает ключ Category.
- Prisma знает о связи Category 1:N Product.

# PostgreSQL и Prisma Schema

| PostgreSQL  | Prisma         |
|-------------|----------------|
| INTEGER     | Int            |
| VARCHAR     | String         |
| BOOLEAN     | Boolean        |
| TIMESTAMPTZ | DateTime       |
| PRIMARY KEY | @id            |
| UNIQUE      | @unique        |
| DEFAULT     | @default(...)  |
| FOREIGN KEY | @relation(...) |

# Таблица PostgreSQL превращается в модель Prisma

Одна и та же структура данных — в базе и в коде приложения



## PostgreSQL

База данных



### products

| # | Колонка     | Тип (пример)   |
|---|-------------|----------------|
| 1 | id          | SERIAL         |
| 2 | name        | TEXT           |
| 3 | price       | NUMERIC(10, 2) |
| 4 | category_id | INTEGER        |
| 5 | in_stock    | BOOLEAN        |

Таблица PostgreSQL

Имена в snake\_case, как обычно в базе данных

PRIMARY KEY → @id

category\_id → @map("category\_id")

products → @@map("products")



## Prisma

Код приложения

```
model Product {
  id      Int      @id
  name    String
  price   Decimal
  categoryId Int    @map("category_id")
  inStock Boolean

  @@map("products")
}
```

Модель Prisma

Имена в camelCase для удобной работы в коде



Имена в **snake\_case** в базе данных могут стать **camelCase** в модели Prisma

# @map и @@map

## PRISMA

```
categoryId Int      @map("category_id")
inStock     Boolean @map("in_stock")
```

```
@@map("products")
```

## Что важно

- @map используется для имени конкретного столбца.
- @@map используется для имени таблицы.
- Это позволяет использовать camelCase в приложении.
- Физические имена PostgreSQL можно оставить в snake\_case.
- Изменение имени модели Prisma не требует переименования таблицы.



# Учебный вопрос 4. Prisma Client

Генерируем API запросов специально для нашей структуры базы

# Что такое Prisma Client

---

- Prisma Client — генерируемый клиент базы данных, методы которого формируются на основе моделей Prisma Schema.
- Для модели Product появляется `prisma.product`.
- Для модели Category появляется `prisma.category`.
- Client знает доступные поля моделей.
- Client предоставляет методы чтения и CRUD.
- После изменения Prisma Schema клиент необходимо генерировать повторно.

## Генерируем Prisma Client

**BASH**

```
npx prisma@7 generate
```

На основе моделей из schema.prisma генерируется Prisma Client с типизированным API для работы с базой данных.

## 1 Модели в schema.prisma

Мы описываем структуру данных, поля и связи.

prisma/schema.prisma

```
model Product {
  id      Int      @id @default(autoincrement())
  name    String   @db.VarChar(150)
  price   Int
  categoryId Int    @map("category_id")
  inStock Boolean @default(true) @map("in_stock")
  category Category #relation(fields: [categoryId],
                    references: [id])
  orderItems OrderItem[]
  @@map("products")
}

model Category {
  id      Int      @id @default(autoincrement())
  name    String   @unique @db.VarChar(100)
  products Product[]
  @@map("categories")
}

model Order {
  id      Int      @id @default(autoincrement())
  userId  Int      @map("user_id")
  status  String   @db.VarChar(30)
  createdAt DateTime @default(now()) @map("created_at")
  items   OrderItem[]
  user    User     @relation(fields: [userId], references: [id])
  @@map("orders")
}
```

 Модели в schema.prisma — это описание существующей или будущей структуры базы данных.

## 2 Запуск генерации

Prisma CLI читает schema.prisma и создаёт клиент.

```
PS C:\project> npx prisma generate
```

```
Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma
```

```
✓ Generated Prisma Client (v7.x.x) to ./generated/prisma
  in 320ms
```



Читает  
модели

**prisma generate**

Создаёт  
Prisma Client

На основе моделей из `schema.prisma`  
генерируется типизированный API.

## 3 Сгенерированный Prisma Client

Готовый API для работы с моделями из schema.prisma.

generated/prisma/client

```
import { PrismaClient } from '../generated/prisma/client'
const prisma = new PrismaClient()

// Работа с моделью Product
const products = await prisma.product.findMany()
const product = await prisma.product.findUnique({
  where: { id: 1 }
})
const newProduct = await prisma.product.create({
  data: { name: 'Keyboard', price: 15000, categoryId: 1 }
})
const updatedProduct = await prisma.product.update({
  where: { id: 1 },
  data: { price: 12000 }
})
await prisma.product.delete({
  where: { id: 1 }
})
```

### ✓ API формируется на основе моделей

- Для каждой модели создаётся свой namespace (prisma.product, prisma.category, prisma.order).
- Доступны типизированные методы: findMany, findUnique, create, update, delete и другие.
- Клиент учитывает поля, типы и связи, указанные в schema.prisma.
- После изменения моделей нужно запустить prisma generate повторно.



## Модели → Генерация → Готовый API

Вы описываете структуру данных, а Prisma создаёт удобный и типизированный клиент.



## Код Prisma Client генерируется автоматически

Файлы в папке generated/prisma изменять вручную не нужно.

# Создаём один экземпляр PrismaClient

## JAVASCRIPT

```
import "dotenv/config";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma/client";

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
});

export const prisma = new PrismaClient({ adapter });
```

## Что важно

- PrismaPg подключает Prisma Client к драйверу pg.
- DATABASE\_URL содержит параметры PostgreSQL.
- Создаётся один экземпляр PrismaClient.
- Другие модули приложения импортируют готовый prisma.
- Не следует создавать новый PrismaClient для каждого HTTP-запроса.

# Получаем все товары

## JAVASCRIPT

```
const products = await prisma.product.findMany({
  orderBy: {
    id: "asc"
  }
});
```

## Что важно

- product соответствует модели Product.
- findMany() возвращает массив объектов.
- SQL SELECT вручную не пишется.
- orderBy определяет сортировку.
- Результат уже является JavaScript-массивом.

# Получаем один товар

## JAVASCRIPT

```
const product = await prisma.product.findUnique({
  where: {
    id: 15
  }
});
```

## Что важно

- findUnique() используется для уникального поля.
- id является PRIMARY KEY и подходит для findUnique().
- where описывает условие поиска.
- Если товар отсутствует, результатом будет null.

# Фильтрация и сортировка

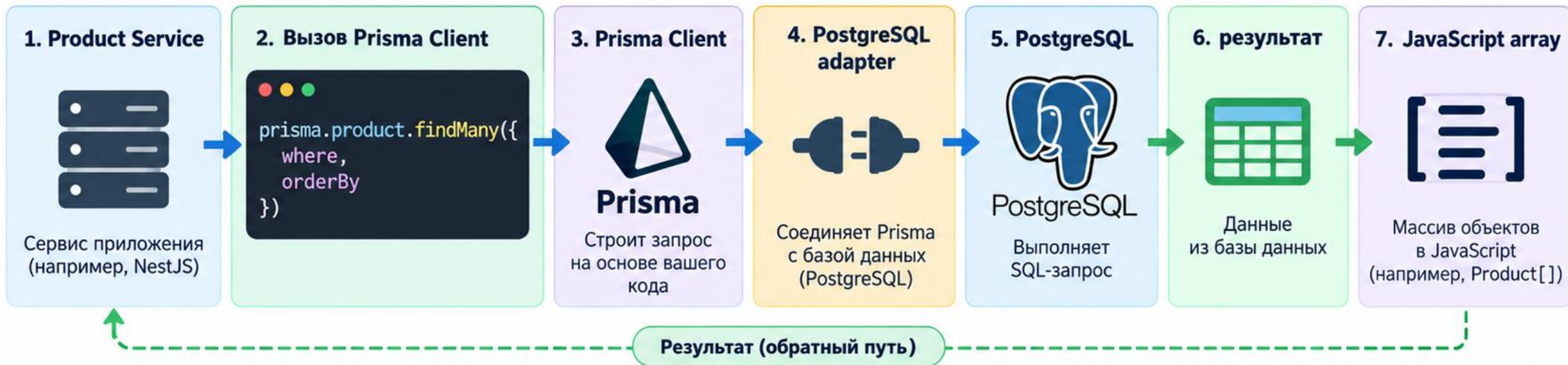
## JAVASCRIPT

```
const products = await prisma.product.findMany({
  where: {
    price: {
      gt: 10000
    },
    inStock: true
  },
  orderBy: {
    price: "asc"
  }
});
```

## Что важно

- where заменяет типичное условие SQL WHERE.
- gt означает greater than.
- Можно комбинировать несколько условий.
- orderBy задаёт сортировку результата.
- Prisma преобразует запрос в операцию PostgreSQL.

# Путь запроса `prisma.product.findMany()` до PostgreSQL



## Эквивалентный SQL (упрощённо)

```
SELECT *  
FROM "Product"  
WHERE ...  
ORDER BY ...;
```



## Разработчик не пишет SQL вручную!

Prisma формирует SQL автоматически на основе вызова `prisma.product.findMany()`.

## Кто за что отвечает?



### Product Service

Вызывает методы ORM (Prisma Client).



### Prisma Client

Строит запрос на основе вашего кода (`prisma.product.findMany()`).



### PostgreSQL adapter

Соединяет Prisma с базой данных (PostgreSQL).



### PostgreSQL

Выполняет SQL-запрос и возвращает данные.



### JavaScript array

Результат приходит в виде массива объектов в JavaScript.



# Учебный вопрос 5. Prisma в Express-приложении

ORM меняет реализацию Service, но не публичный REST API

# Product Service до и после Prisma

## node-postgres

- `pool.query()`
- SQL SELECT вручную
- `$1, $2` для параметров
- `result.rows`
- JOIN пишется вручную

## Prisma

- `prisma.product.findMany()`
- `where`
- `orderBy`
- `findUnique()`
- Работа через модели

# getAllProducts() через Prisma

## JAVASCRIPT

```
export async function getAllProducts() {  
  return prisma.product.findMany({  
    include: {  
      category: true  
    },  
    orderBy: {  
      id: "asc"  
    }  
  });  
}
```

## Что важно

- Service больше не содержит SQL SELECT.
- findMany() получает все товары.
- include временно показывает связанный объект Category.
- Подробная работа со связанными данными будет рассмотрена отдельно.
- Controller не меняется.

## Результат findMany() со связанной категорией Результат • JSON

### Код • JAVASCRIPT

```
const products = await
prisma.product.findMany({
  include: { category: true }
});
```

```
{
  "id": 1,
  "name": "Keyboard",
  "price": 15000,
  "categoryId": 1,
  "inStock": true,
  "category": {
    "id": 1,
    "name": "Accessories"
  }
}
```

### Пояснение

- Prisma Client возвращает обычные JavaScript-объекты. Связанные данные могут быть вложены в результат.

## findProductById() через Prisma

### JAVASCRIPT

```
export async function findProductById(id) {  
  return prisma.product.findUnique({  
    where: { id },  
    include: {  
      category: true  
    }  
  });  
}
```

### Что важно

- Service получает обычное значение id.
- findUnique() ищет товар по PRIMARY KEY.
- SQL WHERE id = \$1 больше не пишется вручную.
- Если товара нет, Prisma возвращает null.
- Controller может преобразовать null в HTTP 404.

# GET /api/products/15 после подключения Prisma

- 1 Route выбирает Controller** GET /api/products/:id передаётся getProductById.
- 2 Controller получает id** req.params.id преобразуется в число 15.
- 3 Controller вызывает Service** findProductById(15) получает обычное JavaScript-значение.
- 4 Service вызывает Prisma Client** prisma.product.findUnique({ where: { id: 15 } }) описывает запрос к модели.
- 5 Prisma обращается к PostgreSQL** ORM формирует и выполняет необходимый запрос через PostgreSQL adapter.
- 6 Controller формирует Response** Найденный объект отправляется как JSON, а null преобразуется в 404.

# Prisma Studio

**BASH**

```
npx prisma@7 studio
```

## Что важно

- Prisma Studio позволяет просматривать данные через браузерный интерфейс.
- Можно открывать таблицы как модели Prisma.
- Удобно проверять результаты лабораторных работ.
- Studio работает непосредственно с подключённой базой.
- Это инструмент разработчика, а не интерфейс конечного пользователя.

## Путь запроса (Request)

1 HTTP Request  
GET /api/products/15

### 2 Router Маршрутизация

```
routes/productRoutes.js

router.get(
 ('/:id',
  getProductById
);
```

### 3 Controller Обработка HTTP

```
controllers/productController.js

export async function
getProductById(req, res) {
  const id = Number(
    req.params.id
  );
  const product = await
  findProductById(id);
  res.json(product);
}
```

### 4 Product Service Бизнес-логика

```
services/productService.js

import { prisma } from
'../config/prisma.js';

export async function
findProductById(id) {
  return prisma.product.
  findUnique({
    where: { id },
    include: { category: true }
  });
}
```

### 5 Prisma Client Доступ к данным

 **Prisma ORM**

- prisma.product.findUnique()
- Формирование SQL-запроса
- Выполнение через адаптер
- Преобразование результата в JavaScript объекты

### 6 PostgreSQL База данных

 PostgreSQL

- ✓ Таблицы (products, categories, orders и др.)
- ✓ Данные
- ✓ Индексы и ограничения
- ✓ Надёжное хранение

## Путь ответа (Response)

8 JSON Response  
200 OK

### 7 Router Передача результата

Результат передаётся назад в middleware и Route

### 6 Controller Формирование ответа

Преобразование данных в JSON и отправка HTTP Response

### 5 Product Service Возврат данных

Получает JavaScript объекты от Prisma и возвращает их в Controller

### 4 Prisma Client Prisma objects

Результат запроса преобразуется в JavaScript объекты (модели)

### 3 Database result Rows from PostgreSQL

PostgreSQL возвращает набор строк (rows)

## Назначение компонентов

### Router Маршрутизация

- Сопоставляет URL с обработчиком
- Извлекает параметры из пути
- Передаёт управление Controller

routes/\*.js

### Controller Обработка HTTP

- Работает с req и res
- Проверяет входные данные
- Вызывает методы Service
- Формирует HTTP-ответ

controllers/\*.js

### Service Бизнес-логика и ORM

- Содержит логику приложения
- Вызывает Prisma Client
- Работает с моделями данных
- Может выполнять валидацию и дополнительные проверки

services/\*.js

### Prisma ORM Доступ к данным

- Предоставляет типизированный API
- Преобразует запросы в SQL
- Возвращает JavaScript объекты
- Работает с моделями из schema.prisma

@prisma/client

### PostgreSQL Хранение данных

- Выполняет SQL-запросы
- Хранит все данные
- Обеспечивает целостность
- Использует индексы и ограничения
- Возвращает результат (rows)

База данных internet shop



Чистая архитектура: маршрутизация → контроллер → сервис → ORM → база данных  
Prisma меняет способ доступа к данным, но не меняет структуру Express-приложения.



REST API остаётся неизменным — меняется только реализация слоя Service.  
Это позволяет постепенно переходить с ручного SQL на Prisma без переписывания контроллеров и роутов.

# Мини-практика: подключите Prisma к shop\_db

Подключите Prisma ORM к существующей PostgreSQL-базе и выполните чтение товаров через Prisma Client.

## Задание

- Установить Prisma ORM 7 и PostgreSQL adapter
- Выполнить `prisma init`
- Настроить `DATABASE_URL`
- Выполнить `prisma db pull`
- Изучить полученные модели
- Выполнить `prisma generate`
- Создать единый экземпляр `PrismaClient`
- Получить товары через `findMany()`
- Получить один товар через `findUnique()`
- Проверить данные через Prisma Studio

## Критерии проверки

- Prisma подключается к существующей `shop_db`
- `schema.prisma` отражает таблицы PostgreSQL
- Prisma Client успешно генерируется
- `findMany()` возвращает список товаров
- `findUnique()` возвращает выбранный товар
- Данные PostgreSQL не удаляются и не создаются заново

# Контрольные вопросы

1. Что означает ORM?
2. Заменяет ли Prisma PostgreSQL?
3. Чем `prisma.product.findMany()` отличается от `pool.query()`?
4. Для чего нужен `schema.prisma`?
5. Что делает `prisma db pull`?
6. Что делает `prisma generate`?
7. Что такое Prisma Client?
8. Для чего используются `@map` и `@@map`?
9. Чем `findMany()` отличается от `findUnique()`?
10. В каком слое Express-приложения следует использовать Prisma Client?

# Итоги занятия

1

ORM предоставляет программный интерфейс для работы с реляционной базой данных.

2

Prisma не заменяет PostgreSQL и не отменяет необходимость понимать SQL.

3

Prisma Schema описывает модели, поля, ключи и связи.

4

`prisma db pull` позволяет получить модели из существующей базы.

5

`prisma generate` создаёт Prisma Client на основе моделей.

6

Prisma Client предоставляет методы `findMany()`, `findUnique()` и другие операции.

7

`@map` и `@@map` позволяют разделить имена моделей приложения от физических имён PostgreSQL.

8

В Express-приложении Prisma располагается внутри слоя Service.

9

REST API может остаться неизменным при переходе с ручного SQL на ORM.